Implementation Walkthrough

Hybrid Memory Benchmark for a Local Student Tutor
A practical explanation, code map and hands-on guide for the 17-step message flow, OCR and response blueprints.
Prepared from the current project files on 8 November 2026.

How to use this report

Explain one step in your own words, run its relevant check, then open the production function. Use the report on screen as a technical walkthrough. The commands demonstrate individual components; do not present a component pass as proof of the whole system or general accuracy.

Working folder

/Users/amitoj/Developer/AITutor

```
cd /Users/amitoj/Developer/AITutor
```

Prerequisites

Start the local frontend, FastAPI and LM Studio with Nomic and Qwen loaded. MySQL and Neo4j must be available for their checks. Tesseract plus the pinned English weights are required for OCR. Do not show database credential files during the review.

One command for the current component bundle

.venv/bin/python -B verification/run_all.py --include-databases

Read the output honestly

PASS means the declared check succeeded. A model connection is not a memory test; a prepared prompt is not a generated answer; cosine similarity is not truth probability; lexical F1 is not factual accuracy. Commands can fail later if a required local service is stopped.

Implementation status

Buffer, extractive rolling summary, vector memory and graph memory are implemented as current local paths. The graph uses rule-based extraction and a labelled keyword fallback. Image support is local OCR, not general vision. Full image-lineage isolation remains future work. One-hour inactivity expiry still limits retention.

Current evidence

The saved four-mode LoCoMo comparison contains 49 distinct questions, three trials per mode, and 588 predictions. Its independent audit passed. The live streaming recall/correction/forget check passed; a separate non-streaming model request timed out at 60 seconds. Those claims have different scopes.

The connected flow

01 Type the question

02 Send to FastAPI

03 Check correction and forgetting commands

04 Save JSON and synchronize MySQL

05 Send text to local Nomic

06 Receive the full embedding

07 Store vector, text and metadata in Chroma

08 Read history, summary, graph and blueprint

09 Create the question search vector

10 Search permitted records using cosine distance

11 Return readable candidates

12 Convert distance to cosine similarity

13 Filter, deduplicate and fit the allowance

14 Assemble Qwen's text prompt

15 Generate through local LM Studio

16 Stream and display the answer

17 Save the completed answer and refresh memory

Two connected model roles

Nomic produces document/query embeddings for search. Qwen generates answers from selected text. LM Studio serves both locally; the backend manages storage, retrieval, prompt construction and mutations.

Two different JSON blueprints

response_blueprint.json supplies trusted response-format instructions and rolling-summary policy. memory_blueprint.json structures session records. Neither is the Nomic model or the Chroma search algorithm.

Keep LoCoMo separate

The offline benchmark supplies conversations, questions, references and scoring. It does not automatically run after every real chat, and it does not provide another model that replaces your local Qwen.

01 Type the question

Explain it

The React interface collects the question. It calls sendMessage(), which prepares the active chat and forwards the content. The browser is the interface; it does not run Nomic or Qwen.

Purpose

Keep interaction and display separate from model and memory processing.

Input and output

Input: typed text and optional attachment. Output: a frontend send operation for the selected learner/chat.

Open the implementation

frontend/src/App.tsx:136

frontend/src/components/MessageComposer.tsx:1

Code to recognize

```
async function sendMessage(content: string, image?: File) {
  // Existing frontend handler: see the referenced source.
}
```

Function signature from App.tsx; body intentionally omitted here. Open the source for the implementation.

Try it yourself

Open http://127.0.0.1:5173/ and create a fresh presentation chat. Type: Explain binary search in Python.

What to observe

The message appears in the selected chat. This establishes UI submission, not that a model generated the answer.

Likely question

Does React itself generate the answer?

Answer

No. It sends the request and renders returned events.

Failure to explain

A blank UI, disabled send button, or failed request can be a frontend/network issue before any model is called.

Design trade-off

Separating UI and backend adds a local network boundary, but makes each part easier to inspect.

02 Send to FastAPI

Explain it

React sends a POST request containing the message to our local FastAPI endpoint. The learner ID is sent in X-User-Id, and the chat ID is part of the route. FastAPI passes the work to ChatService.stream_message().

Purpose

Provide the request boundary, validate data and identify which learner/chat owns the operation.

Input and output

Input: HTTP request. Output: a streaming response produced by the Python service.

Open the implementation

frontend/src/api/chatApi.ts:77

backend/api/chat.py:81

backend/services/chat_service.py:157

Code to recognize

```
headers: {
  "Content-Type": "application/json",
  "X-User-Id": userId,
},
body: JSON.stringify(body),
```

Excerpt from the real streamRequest() request construction.

Try it yourself

```
.venv/bin/python -B verification/inspect_live_chat.py --follow
```

Open the browser Developer Tools, choose Network, then send a message. Inspect POST /chat/{chat_id}/message/stream and its request payload.

What to observe

Show the local URL, the content field and the text/event-stream response. Do not display personal headers from unrelated chats.

Likely question

Does a learner ID header provide secure authentication?

Answer

No. It scopes records in this local prototype; production authentication would require more controls.

Failure to explain

HTTP errors or missing stream events can stop the flow here. A reachable backend is not proof that model inference succeeded.

Design trade-off

Explicit IDs support ownership filters. A client-supplied ID alone is not a production identity system.

03 Check correction and forgetting commands

Explain it

Before normal generation, the backend checks supported memory commands. A forget request triggers actual source/index cleanup and a backend acknowledgement. We do not rely on Qwen merely saying it has forgotten.

Purpose

Make memory changes enforceable in storage rather than conversational promises.

Input and output

Input: message text or explicit patch/delete request. Output: a mutation result, clarification, or normal question processing.

Open the implementation

backend/services/chat_service.py:166

backend/memory/mutations.py:218

backend/memory/mutations.py:92

backend/api/chat.py:154

Code to recognize

```
control = self._repository.mutations.control(
    payload.content, user_id, chat.id
)
```

The real command check. The normal generation branch and command branch then differ.

Try it yourself

```
.venv/bin/python -B verification/verify_mysql.py
```

Run the isolated MySQL and Neo4j checks. For the full local-model streaming regression, use scripts/validate_current_tutor.py. For a manual demo, forget only a fresh presentation chat.

What to observe

The source is removed or corrected across enabled stores. An ambiguous request should ask for clarification rather than claim deletion.

Likely question

Does deleting JSON alone make the chatbot forget?

Answer

No. Retained summaries, vectors, graph records or recent source copies can still supply the fact.

Failure to explain

Cleanup can fail. The implementation keeps a pending operation and should not acknowledge successful forgetting until cleanup succeeds.

Design trade-off

Coordinated cleanup is more work than a prompt instruction. Backups, exports and old provider logs remain outside the active-store deletion boundary.

04 Save JSON and synchronize MySQL

Explain it

The repository stores the message by its unique ID in the JSON working store. With MySQL enabled, a pending operation coordinates the JSON write and SQL mirror. MySQL supplies structured persistence; Qwen never receives a database connection.

Purpose

Keep original source text and ownership available independently of embeddings.

Input and output

Input: a Message record. Output: saved JSON plus synchronized SQL records, or a retryable pending operation.

Open the implementation

backend/database/chroma_repository.py:99

backend/database/mysql_repository.py:77

backend/database/mysql_repository.py:49

backend/database/mysql_store.py:72

Code to recognize

```
messages = self._read_json_file(self._message_file)
messages[message.id] = message.model_dump_json()
self._write_json_file(self._message_file, messages)
```

Actual repository save operations. In the MySQL subclass, _write_json_file() invokes the mirror/recovery path.

Try it yourself

```
.venv/bin/python -B verification/verify_mysql.py
```

Run verify_mysql.py. It saves a temporary source, reads it back from SQL, patches it through FastAPI, then deletes it and checks SQL again.

What to observe

SQL SAVED, SQL UPDATED and SQL SOURCE DELETED refer to real test records in an isolated namespace.

Likely question

Why keep JSON and MySQL?

Answer

This project currently uses JSON as working memory and MySQL as a synchronized persistence/recovery mirror. The extra copy has a consistency cost.

Failure to explain

A SQL outage can leave pending synchronization. The local recovery mechanism is not a distributed atomic transaction across all databases.

Design trade-off

Structured persistence and recovery are useful, but keeping two stores increases synchronization complexity.

05 Send text to local Nomic

Explain it

In vector or hybrid mode, saved searchable text is sent to Nomic through LM Studio on the local machine. The embedding request uses search_document for stored text. Nomic computes the vector; it is not the conversation database.

Purpose

Create a representation that can be searched by semantic similarity rather than only matching exact words.

Input and output

Input: text with the document prefix. Output: an HTTP embedding response from local LM Studio.

Open the implementation

backend/memory/semantic_chat.py:23

backend/main.py:72

backend/config.py:58

Code to recognize

```
prefix = "search_query: " if query else "search_document: "
response = httpx.post(
    f"{self.base_url}/embeddings",
    json={"model": self.model,
          "input": [prefix + text for text in texts]},
    timeout=60, trust_env=False,
)
```

Actual embed() request construction. The model and base URL are configured by the app.

Try it yourself

```
.venv/bin/python -B verification/verify_nomic_embeds.py
```

Open LM Studio Developer Logs, then run verify_nomic_embeds.py. Find the unique VERIFY_NOMIC marker in the request.

What to observe

The document request goes to /v1/embeddings. The same check separately demonstrates a search_query request.

Likely question

Why do we need LM Studio here?

Answer

It loads and serves the local Nomic model. Our Python backend sends the request and manages the returned data.

Failure to explain

An unloaded/missing model, stopped server, bad model identifier or timeout can prevent indexing. JSON can be saved while semantic indexing needs retry.

Design trade-off

Local inference keeps model requests on loopback, but shares this machine's memory and compute.

06 Receive the full embedding

Explain it

Nomic returns one vector per supplied text input. In our verified configuration, a vector has 768 numerical coordinates. The verification script displays only the first eight; the real pipeline retains the complete vector.

Purpose

Validate the response before indexing it and preserve the whole representation.

Input and output

Input: embedding response JSON. Output: ordered, validated vectors.

Open the implementation

backend/memory/semantic_chat.py:30

verification/verify_nomic_embeds.py:1

Code to recognize

```
data = sorted(response.json()["data"],
              key=lambda row: row["index"])
vectors = [row["embedding"] for row in data]
```

Actual extraction of complete vectors. vector[:8] exists only in the display check.

Try it yourself

```
.venv/bin/python -B verification/verify_nomic_embeds.py
```

Run the Nomic check and compare Returned dimensions with First 8 coordinates. Open semantic_chat.py and locate return vectors.

What to observe

768 means coordinates in one vector, not 768 separate embeddings or 768 words. Individual coordinates are not named facts such as Python or university.

Likely question

Can we convert that vector back into the exact original sentence?

Answer

We do not do that. We preserve original text separately and retrieve it by record ID.

Failure to explain

The code rejects missing, zero, non-finite or count-mismatched vectors. A valid shape alone does not prove good semantic retrieval.

Design trade-off

Dense vectors make similarity search possible but do not preserve a readable explanation of every coordinate.

07 Store vector, text and metadata in Chroma

Explain it

The semantic index links each complete vector to its original text and message ID. Metadata includes learner, chat, role, timestamp and a content hash. This is why a vector match can return a readable source message.

Purpose

Connect numerical retrieval to traceable source records.

Input and output

Input: vectors, text, IDs and metadata. Output: inserted or updated Chroma records.

Open the implementation

backend/memory/semantic_chat.py:18

backend/memory/semantic_chat.py:57

backend/database/chroma_repository.py:107

Code to recognize

```
self.collection.upsert(
    ids=[m.id for m in batch],
    documents=[searchable(m) for m in batch],
    embeddings=vectors,
    metadatas=[...],
)
```

Actual upsert structure; metadata details are abbreviated. embedding_function=None means our code explicitly supplies the Nomic vectors.

Try it yourself

```
.venv/bin/python -B verification/verify_chroma_storage.py
```

Run verify_chroma_storage.py. Follow one message ID through TEXT, METADATA, STORED VECTOR DIMENSIONS and FIRST 8.

What to observe

The same record ID connects those values. The demo uses actual temporary Chroma storage and actual Nomic embeddings.

Likely question

Does Chroma automatically train Qwen with these records?

Answer

No. It stores/searches records. Model weights are not updated by an upsert.

Failure to explain

An incorrect collection/model pairing or failed synchronization can make records missing or unsuitable for comparison. The collection name includes a hash of the configured embedding-model identifier.

Design trade-off

The vector index adds fast search and provenance, but must be reconciled when source text changes or is deleted.

08 Read history, summary, graph and blueprint

Explain it

The backend starts context preparation by selecting a response template and loading this chat's history. Depending on the configured mode, it can also read an extractive rolling summary and Neo4j evidence. Databases provide source material; the response blueprint provides instructions.

Purpose

Combine recent conversation and enabled older memory without sending the whole archive.

Input and output

Input: question, learner/chat scope and enabled mode. Output: candidate history, summary, graph facts and trusted format instructions.

Open the implementation

backend/services/chat_service.py:307

backend/services/chat_service.py:337

backend/memory/rolling_summary.py:23

backend/memory/graph_memory.py:164

backend/services/response_blueprint.py:23

Code to recognize

```
template = select_template(user_message,
                           self._response_blueprint)
# _prepare_context() then reads enabled memory paths.
```

Actual selection statement; memory-reading branches are in the referenced method.

Try it yourself

```
.venv/bin/python -B verification/verify_rolling_summary.py
```

Run verify_blueprint.py and verify_rolling_summary.py. Run verify_neo4j.py if both local databases are available.

What to observe

Template instructions appear in the actual prompt; summary excerpts and source IDs are persisted. Graph facts point to supporting user messages.

Likely question

Is rolling summary the same as asking the bot to summarize?

Answer

No. Our rolling-memory baseline automatically selects bounded older source excerpts. A user-requested conversational summary is a separate generation task.

Failure to explain

A useful summary can still be excluded by the input allowance. We observed that at the older 2,800-unit setting. Graph coverage is limited by its extraction/query rules.

Design trade-off

Each memory approach preserves different information. None should be labelled a full reproduction of MemGPT, Mem0 or A-MEM.

09 Create the question search vector

Explain it

For semantic retrieval, the backend sends the current question to the same Nomic embedding model using search_query. The new question vector and stored document vectors are therefore produced by the compatible model configuration.

Purpose

Represent what we are looking for in the same vector space as searchable source records.

Input and output

Input: the current question. Output: its full search embedding.

Open the implementation

backend/memory/semantic_chat.py:86

backend/memory/semantic_chat.py:91

Code to recognize

```
self.sync(message.user_id)
embedding = self.embed(
    [message.content], query=True
)[0]
```

Actual retrieve() sequence: reconcile the learner's index, then embed the search question.

Try it yourself

```
.venv/bin/python -B verification/verify_nomic_embeds.py
```

Run verify_nomic_embeds.py and show the search_query input. The cosine check then uses a question about binary-search difficulty.

What to observe

A question can be saved as a document and embedded again for searching; those operations have different roles and prefixes.

Likely question

Why can't we compare a vector from any other model?

Answer

Different embedding models need not share compatible dimensions or geometry. The index and query should use the same configured embedding space.

Failure to explain

A stale index or incompatible model can spoil retrieval even when the question embedding succeeds.

Design trade-off

Query embedding adds latency on each search, but enables meaning-based retrieval.

10 Search permitted records using cosine distance

Explain it

Chroma searches vectors in a cosine-distance collection. Ownership filters restrict candidates, and the application reserves slots for user and assistant records. Cosine compares the complete vector directions; it does not compare one coordinate such as 0.7 with 0.8.

Purpose

Find related source messages while restricting which records are eligible.

Input and output

Input: question vector and filters. Output: nearest candidates with IDs, source text, metadata and distances.

Open the implementation

backend/memory/semantic_chat.py:95

backend/memory/semantic_chat.py:98

backend/memory/semantic_chat.py:20

Code to recognize

```
result = self.collection.query(
    query_embeddings=[embedding],
    n_results=limit + 1,
    where={"$and": filters},
    include=["documents", "metadatas", "distances"],
)
```

Actual search call. Chroma handles its indexed search internally; we should not describe our code as manually scanning every vector.

Try it yourself

```
.venv/bin/python -B verification/verify_cosine.py
```

Run verify_cosine.py. Read each ranked example and its original text before looking at the number.

What to observe

For the checked example, similarity agrees numerically with an independent dot-product/norm calculation. Scores are not per-word matches.

Likely question

Is Chroma running an LSTM?

Answer

No. It searches embeddings already generated by Nomic. An LSTM is a different sequence-processing model.

Failure to explain

A nearby vector can be related to the topic but miss the specific fact or multiple supporting turns needed to answer.

Design trade-off

Semantic retrieval handles paraphrases but does not guarantee evidence completeness or factual truth.

11 Return readable candidates

Explain it

Chroma returns the IDs, original document text, metadata and distances for candidate records. The backend can inspect the readable source because the vector was stored alongside that source under a message ID.

Purpose

Bring numerical search results back into traceable text evidence.

Input and output

Input: Chroma query response. Output: candidate dictionaries and retrieval diagnostics.

Open the implementation

backend/memory/semantic_chat.py:104

backend/memory/semantic_chat.py:112

Code to recognize

```
for key, document, metadata, distance in zip(
    result["ids"][0], result["documents"][0],
    result["metadatas"][0], result["distances"][0],
):
    # Actual retrieve() loop; see the source for its body.
```

Actual result fields and loop signature. The complete body attaches the similarity and metadata; it excludes the current message ID.

Try it yourself

```
.venv/bin/python -B verification/verify_cosine.py
```

Run verify_cosine.py or inspect a completed real chat with inspect_live_chat.py. Follow a returned record ID back to its document text.

What to observe

A candidate contains readable text, role, ownership and timestamp information. Returned candidates are not necessarily the final prompt contents.

Likely question

Is Chroma decoding the vector back into a sentence?

Answer

No. It returns the original text already stored alongside the vector.

Failure to explain

The returned message can be related but insufficient, or it can be a wrong earlier assistant answer. Source role and the actual text matter.

Design trade-off

Storing source text with the index improves traceability but duplicates content and requires consistent deletion.

12 Convert distance to cosine similarity

Explain it

Chroma returns cosine distance and linked source text. Our code computes similarity as one minus that distance. The result helps choose candidates; it is not a calibrated probability that an answer will be correct.

Purpose

Expose readable retrieval evidence and a usable similarity measure.

Input and output

Input: Chroma search results. Output: candidate dictionaries and a retrieval trace.

Open the implementation

backend/memory/semantic_chat.py:104

backend/memory/semantic_chat.py:110

verification/verify_cosine.py:1

Code to recognize

```
"cosine_similarity": round(1 - distance, 6)
```

*Actual conversion. The verification script independently computes dot(q,m)/(norm(q)*norm(m)).*

Try it yourself

```
.venv/bin/python -B verification/verify_cosine.py
```

Run verify_cosine.py and compare manual cosine against 1-distance. Explain the dot product and both vector norms printed by the script.

What to observe

A saved test produced manual 0.74160495 versus Chroma-derived 0.74160498. That small difference is within the check's floating-point tolerance; reruns may print slightly different values.

Likely question

Does a similarity of 0.74 mean 74% correctness?

Answer

No. It describes vector similarity for that query/message pair. It does not assess the generated answer.

Failure to explain

Similar scores do not prove both messages are useful. One can be about an irrelevant hobby or lack the required detail.

Design trade-off

The score is convenient for filtering, but relevance should be inspected and thresholds validated on separate data.

13 Filter, deduplicate and fit the allowance

Explain it

The backend keeps recent history, checks vector candidates against the configured cutoff, avoids duplicate source IDs and includes older text only when space remains. Retrieval and prompt inclusion are separate decisions.

Purpose

Use bounded model input without repeatedly inserting the same source.

Input and output

Input: history, retrieved candidates and remaining allowance. Output: selected messages/evidence with inclusion or exclusion flags.

Open the implementation

backend/services/chat_service.py:424

backend/services/chat_service.py:442

backend/memory/semantic_chat.py:120

Code to recognize

```
if item["cosine_similarity"] < self._memory_min_similarity:
    item["exclusion_reason"] = "below_similarity_threshold"
    continue
if item["id"] in selected_ids:
    item["included_as_history"] = True
    continue
```

Actual cutoff and duplicate checks. The configured cutoff in our evaluated run was 0.60; it is a tunable setting, not a universal optimum.

Try it yourself

```
.venv/bin/python -B verification/verify_context.py
```

Run verify_context.py. Compare included_in_prompt, included_as_history and exclusion_reason for each candidate.

What to observe

A candidate can be excluded as extra vector evidence because it is already present in history. An answer based on that source then does not prove older-message vector retrieval was necessary.

Likely question

Is the 6,144 allowance the actual tokenizer count?

Answer

No. The helper counts UTF-8 bytes plus a small formatting allowance. LM Studio separately reports actual model tokens.

Failure to explain

Budget selection can omit the required fact, cut an excerpt, or spend space on unrelated text. A model's maximum context window is a separate limit.

Design trade-off

More context can preserve evidence but increases processing cost and can add distracting information.

14 Assemble Qwen's text prompt

Explain it

The backend combines tutor instructions, the selected response blueprint, recent messages, admitted older evidence and the current question. This assembled text is what Qwen receives. Nomic vectors stay in the retrieval layer.

Purpose

Give the generator relevant evidence and clear instructions in a bounded request.

Input and output

Input: selected sources and trusted template. Output: system prompt plus ordered role/content messages.

Open the implementation

backend/services/chat_service.py:461

backend/services/chat_service.py:465

backend/services/response_blueprint.py:9

response_blueprint.json:1

Code to recognize

```
if evidence:
    system += "\nRetrieved conversation evidence:" + "".join(evidence)
messages = [*selected, user_message]
```

Actual final vector-evidence insertion and message assembly. Graph grounding can also be attached earlier in the same method.

Try it yourself

```
.venv/bin/python -B verification/verify_context.py
```

Run verify_context.py to print the prepared prompt. Run verify_blueprint.py to prove that template instructions were inserted.

What to observe

Show the actual source text inside the prompt, not just a list of retrieved IDs. The context check prepares the request but does not send it to Qwen.

Likely question

Does the blueprint guarantee a correct answer?

Answer

No. It is an instruction template. Model compliance and factual correctness must be evaluated separately.

Failure to explain

Qwen may ignore, misread or overgeneralize supplied evidence. Source text is treated as quoted data, not trusted instructions.

Design trade-off

Explicit grounding is explainable, but longer prompts cost more and can still confuse a small model.

15 Generate through local LM Studio

Explain it

ChatService sends the assembled request through the configured model adapter. In our local setup the destination is LM Studio, which runs Qwen2.5 1.5B Instruct. Qwen generates text from its trained weights and the provided prompt; the request does not train it.

Purpose

Perform local language generation without giving Qwen direct database access.

Input and output

Input: text prompt and generation settings. Output: generated response chunks or an error.

Open the implementation

backend/services/chat_service.py:259

backend/main.py:66

backend/llm/groq_client.py:113

backend/llm/groq_client.py:155

Code to recognize

```
yield from self._llm_client.stream_reply(
    messages, system_prompt=system_prompt
)
```

Actual streaming call. GroqClient is a historical class name; backend/main.py supplies the local base URL.

Try it yourself

```
.venv/bin/python -B verification/verify_qwen.py
```

Open LM Studio Developer Logs and run verify_qwen.py. Match the VERIFY_QWEN marker and read the actual returned response.

What to observe

The destination is /v1/chat/completions on loopback. LOCAL_QWEN_OK demonstrates a short model response, not tutoring accuracy.

Likely question

Does LM Studio automatically make Qwen smarter or faster?

Answer

It serves and executes the model. Speed depends on hardware, loading/offload settings, prompt/output lengths and other workload; memory retrieval does not change parameter count.

Failure to explain

A separate non-streaming live check hit its 60-second timeout. The streaming check passed, but a long first answer took about 51 seconds to finish.

Design trade-off

Local privacy and control come with finite RAM/compute and weaker performance than some larger-model setups.

16 Stream and display the answer

Explain it

FastAPI emits server-sent events containing answer chunks. React appends these chunks and renders the content. Markdown fences become code panels with a language label and copy button; the browser does not execute that code.

Purpose

Show output progressively while keeping rendering separate from generation.

Input and output

Input: streamed text events. Output: visible response and final message state.

Open the implementation

backend/services/chat_service.py:204

backend/services/chat_service.py:553

frontend/src/api/chatApi.ts:123

frontend/src/App.tsx:190

frontend/src/components/MessageBubble.tsx:52

Code to recognize

```
yield self._sse("chunk", {"content": chunk})
# Frontend event handler:
handlers.onChunk((parsed.data as { content: string }).content);
```

Real Python emission and TypeScript receipt, shown together to connect the two files.

Try it yourself

```
.venv/bin/python -B verification/inspect_live_chat.py --follow
```

Open the browser Network response for /message/stream while sending a short question. Follow chunk events, then the done event. Use inspect_live_chat.py after completion.

What to observe

Streaming can display some text before the complete request ends. The saved completion and done event are the final state; a partial chunk alone is not success.

Likely question

Is request latency the same as time to the first visible token?

Answer

No. Our saved live-route timing measured the completed response, including retrieval and persistence. First-token latency needs separate instrumentation.

Failure to explain

A disconnected or failed stream can leave visible partial text without a persisted completed answer. Browser scrolling/formatting bugs are frontend issues, not embedding failures.

Design trade-off

Progressive display improves interaction, but requires explicit event parsing, cancellation and error handling.

17 Save the completed answer and refresh memory

Explain it

After generation completes, the backend checks whether memory changed during the answer, saves the completed reply and updates enabled derived stores. The response can be searchable later, but an assistant claim is not automatically a verified learner fact.

Purpose

Close the conversation loop while guarding against restoring information that was just forgotten.

Input and output

Input: completed answer, revision and source scope. Output: persisted reply, synchronized enabled memory and a final done event.

Open the implementation

backend/services/chat_service.py:214

backend/services/chat_service.py:247

backend/database/chroma_repository.py:99

backend/memory/graph_memory.py:98

backend/memory/graph_memory.py:35

Code to recognize

```
self._check_revision(user_id, revision)
self._repository.save_message(assistant_message)
```

Actual revision check and save in the streaming completion path. save_message() invokes enabled summary, vector and graph reconciliation.

Try it yourself

```
.venv/bin/python -B verification/verify_neo4j.py
```

Run verify_chroma_storage.py, verify_neo4j.py and verify_mysql.py for the component paths. Use scripts/validate_current_tutor.py for the real streaming recall/correction/forget check with local models.

What to observe

The saved live check recalled learner facts, changed the name to AJ, removed the source from JSON/MySQL/Chroma/Neo4j on forgetting, and did not recall the old topic afterward.

Likely question

Can storing assistant answers pollute later retrieval?

Answer

Yes. Wrong generated text can be retrieved later. Source roles and original user statements matter; our graph fact extraction uses supported user assertions, not assistant claims.

Failure to explain

One store can be unavailable, and source synchronization must retry. A local revision guard is not proof against every distributed race or every leaked backup.

Design trade-off

Retaining generated replies supports continuity but creates a feedback-pollution risk. Full correctness requires more than successful persistence.

OCR and blueprint: the image path

1. React uploads the selected image bytes to POST /chat/{chat_id}/images. The route checks ownership, supported format, nonempty content and the 25 MiB bound.

2. FastAPI runs LocalOCR.extract() in a thread pool. Tesseract uses the local pinned English LSTM weights; the OCR engine is separate from Nomic, Qwen and LM Studio.

3. OCR returns readable text, word records, engine confidence scores, extraction time and the model checksum. OCR confidence is not factual confidence.

4. SessionMemory.add_image() copies the file into managed local storage and saves image metadata and full OCR in the common session record.

5. React sends the returned image ID with the chat question. ChatService._attach_image() resolves that ID for the owning learner/chat and attaches an OCR excerpt of up to 1,000 UTF-8 bytes.

6. select_template() checks image_attachment first, so image_explanation is selected before the summary/normal-text alternatives.

7. _image_context() quotes the OCR excerpt in the question. _prepare_context() adds the image-specific instruction and warns that the model has not seen the pixels.

8. Qwen answers from the supplied text through the normal local generation path. _record_image_answer() stores the associated question/answer in the image record.

Source locations

backend/api/chat.py:103; backend/ocr/local_ocr.py:72; backend/memory/session_memory.py:106; backend/services/chat_service.py:489; backend/services/response_blueprint.py:23.

What images can this explain?

Readable screenshot or document text, subject to OCR and model errors. Objects, diagrams, handwriting and equations are not reliably solved by this text-only path. Do not describe OCR as adding full vision to Qwen.

Image memory boundary

The searchable message may contain OCR text and be embedded by Nomic. Strict isolation of image-derived history across all summaries, graph facts and follow-ups is not yet implemented. Do not promise that merely switching topics excludes every image-derived record.

OCR: run, inspect and explain

```
.venv/bin/python -B verification/verify_ocr_blueprint.py
```

What this command actually calls

LocalOCR.extract() -> SessionMemory.add_image() -> ChatService._attach_image() -> select_template() -> _prepare_context(). It uses the existing binary-search screenshot fixture and temporary managed storage.

What to show

Show the OCR engine/language, extracted text, returned word count and latency, managed image ID, image_explanation template instruction, and actual OCR text inserted into the prepared question.

What it does not show

This check does not call Qwen, perform vector indexing, or run the browser upload route. Its prepared question shows the text that generation would receive. Use the frontend image upload for the full interaction.

Recognition error from the actual check

The fixture produced the line "Which programming algorithm was | learning?". The vertical bar illustrates an OCR recognition error. Read the extracted text before trusting an explanation built from it.

Blueprint distinction

normal_content requests a brief definition, steps, example and takeaway for explanations. conversation_summary requests supported Topics, Learner needs and Next steps. image_explanation requests a step-by-step explanation of readable OCR text with uncertainty stated. The shared code-format instruction requests language-labelled Markdown fences.

Likely question

Is the image template an enforced JSON output schema?

No. It is a trusted natural-language system instruction selected by backend code. It guides output; it does not force factual correctness or guarantee format compliance.

Safe manual demo

Upload a clear text screenshot into a fresh presentation chat. Expand its extracted text, ask a specific question about that text, and show the local OCR path and selected template. Avoid private images and unrelated stored conversations.

LoCoMo: how the evaluation works

Source and model

The public LoCoMo conversations, questions and references come from https://github.com/snap-research/locomo. Your local Qwen generates the predictions using your current context builder; the dataset is not a replacement model.

Protocol

Load only source dialogue and supplied captions into isolated memory. Build context in each mode. Generate an answer locally. Only afterward compare it with the withheld reference using upstream scoring. Do not write predictions or answer keys back into test memory.

Scope

49 unique questions from all 10 conversations, selected deterministically by category. Four modes and three trials produce 588 predictions. This is a fixed subset, not all 1,986 questions. Offline context-builder evaluation and the separate full streaming-route check have different scopes.

Memory	Mean answer F1, 1-4	Abstention, 5
buffer	0.0053	0.9000
summary	0.0098	0.8333
vector	0.1637	0.6000
graph	0.0247	0.6000

Interpretation

F1 for categories 1-4 measures normalized/stemmed wording overlap. Category 5 uses the upstream abstention phrase rule. Neither is a universal factual-accuracy percentage. Vector had the strongest mean F1 here, but real wrong and incomplete answers remain.

Actual scoring limitation

Qwen predicted 2023-05-07 against reference 7 May 2023 and got lexical F1 0.0. The dates are equivalent. Preserve the official score and explain the mismatch separately; do not silently rewrite scoring to make results look better.

Latency

Context construction, local generation request time and full UI-route completion time are different. The 6,144 input allowance is a conservative byte bound, not tokenizer tokens. Fixed order, caching and active workstation load limit latency comparisons.

```
.venv/bin/python -B scripts/show_evaluation_results.py --answers
```

Terms and boundaries

Message

A stored user or assistant record, with an ID and ownership. It may contain several sentences.

Embedding

One numerical vector for a supplied text input. In this Nomic setup it has 768 coordinates.

Semantic search

Search by vector similarity. Related wording is not proof of complete or correct evidence.

Cosine distance / similarity

Chroma returns cosine distance. Current code reports similarity as 1-distance. It is not a calibrated correctness score.

Context / prompt

The selected text and instructions supplied for one generation request. It is not all database contents.

Token / input allowance

Model tokens are tokenizer units. The current budget helper instead uses UTF-8 bytes plus a formatting allowance.

Rolling summary

The automatic bounded extractive memory baseline; separate from asking Qwen to summarize a conversation.

Response blueprint

A trusted array of response-format templates plus policy. It is not a model or a database schema.

SSE / streaming

Server-sent events deliver chunks and a final completion event over HTTP.

OCR

Local text recognition from image pixels, currently Tesseract with English LSTM weights. Not general visual understanding.

Source / provenance

The original message and identifier supporting a retrieved excerpt or extracted fact.

Correction / forgetting

Backend mutations to active source and derived records; different from Qwen agreeing conversationally.

Local privacy

Model requests are configured for loopback. This does not certify authentication, backups, provider logs or every possible threat.

Cross-session retention

Only retained sources are eligible. Current one-hour inactivity expiry can remove them; LM Studio model-unload TTL is a separate setting.

Demonstrate it in front of the panel

Introduce the report

"This is a technical walkthrough of our implementation. I will explain one stage, run its verification, and show the function responsible." Use the document on screen if your panel permits reference material; avoid reading paragraphs word for word.

A practical order

1. Show the browser request.
2. Run Nomic and match the log marker.
3. Show the stored Chroma record ID and vector.
4. Run cosine and explain the independent calculation.
5. Show the selected prompt and its inclusion flags.
6. Show Qwen's local response.
7. Demonstrate correction/forgetting in isolated storage.
8. Show OCR and the selected image blueprint.
9. Present LoCoMo results with one actual failure.

When asked for code

Open the production source named by the check. Point to the input, the operation and the return value. The verification folder calls that implementation; it is not a second chatbot and not a substitute for understanding the backend.

When something fails

State the observed error, identify the stage and inspect its evidence. Do not invent a reason or call a partial response a completed save. A stopped local model server is different from missing retrieval evidence or a wrong generated fact.

What you can confidently say

The component bundle and added OCR check have recorded passes. The independent 588-prediction scoring audit passed. A real local streaming check passed learner recall, correction and active-store forgetting. These findings do not establish perfect answer correctness.

What remains limited

Wrong facts and incomplete multi-hop answers; weak summary/graph QA on this subset; lexical scoring sensitivity; long-response latency and a non-streaming timeout; incomplete image-lineage gating; no guarantee that external exports/backups have been forgotten.

Your own practice record

For each check, write down: command, input, function, output, what passed, and one limitation. Be able to explain one source message moving through the chain without reciting all 17 sections.